



преврати модель в параметризованную Python-функцию

Ниже — параметризованная Python-функция для твоей toy-model. Она реализует 5D-конусоподобную геометрию $a(\rho) = a_0 - k\rho$, различающиеся эффективные скорости Γ_B и $\Gamma_{\bar{B}}$, и вычисляет асимметрию $A(\rho) = \frac{n_B - n_{\bar{B}}}{n_B + n_{\bar{B}}}$ [1].

Код

```
import numpy as np

def baryon_asymmetry_model(
    rho,
    a0=1.0,
    k=0.5,
    epsilon0=0.02,
    alpha=1.0,
    gamma0=1.0,
    delta0=0.0,
    n0=1.0,
    a_min=1e-9,
    phi0=1.0,
    phi_beta=0.0,
    use_phi=False,
    source_B=None,
    source_Bbar=None,
    return_all=True,
):
    """
    Toy-model of geometry-driven baryon asymmetry in a 5D cone-like background.

    Parameters
    -----
    rho : array-like
        Monotonic coordinate along the cone / evolution parameter.
    a0 : float
        Initial geometric scale factor a(rho=0).
    k : float
        Cone narrowing rate in a(rho) = a0 - k*rho.
    epsilon0 : float
        Baseline CP/C asymmetry strength.
    alpha : float
        Power controlling how asymmetry grows as a(rho) shrinks.
    gamma0 : float
        Symmetric damping / annihilation rate.
```

```

delta0 : float
    Initial asymmetry in densities:
    n_B(0)=n0(1+delta0), n_Bbar(0)=n0(1-delta0).
n0 : float
    Initial baseline density.
a_min : float
    Lower cutoff on a(rho) to avoid singularities.
phi0 : float
    Extra-dimension scale prefactor.
phi_beta : float
    Extra-dimension scaling exponent in phi(rho)=phi0*a(rho)^phi_beta.
use_phi : bool
    If True, asymmetry term uses (a*phi)^(-alpha) instead of a^(-alpha).
source_B, source_Bbar : None, scalar, or callable
    Optional source terms S_B(rho), S_Bbar(rho) in:
    dn/d rho = -Gamma*n + S
return_all : bool
    If True, return detailed dictionary; otherwise return A(rho) only.

Returns
-----
dict or np.ndarray
"""
rho = np.asarray(rho, dtype=float)

if rho.ndim != 1 or rho.size < 2:
    raise ValueError("rho must be a 1D array with at least 2 points")
if np.any(np.diff(rho) <= 0):
    raise ValueError("rho must be strictly increasing")
if a0 <= 0 or a_min <= 0 or n0 < 0 or gamma0 < 0:
    raise ValueError("Require a0>0, a_min>0, n0>=0, gamma0>=0")
if abs(delta0) >= 1:
    raise ValueError("delta0 must satisfy |delta0| < 1")

# Geometry
a = np.maximum(a0 - k * rho, a_min)
phi = phi0 * np.power(a, phi_beta)

# Geometry-controlled asymmetry kernel
geom_kernel = np.power(a * phi if use_phi else a, -alpha)
delta_gamma = epsilon0 * geom_kernel

# Effective rates
Gamma_B = gamma0 - delta_gamma
Gamma_Bbar = gamma0 + delta_gamma

# Initial conditions
n_B = np.zeros_like(rho)
n_Bbar = np.zeros_like(rho)
n_B[0] = n0 * (1.0 + delta0)
n_Bbar[0] = n0 * (1.0 - delta0)

def _src(val, x):
    if val is None:
        return 0.0
    if callable(val):

```

```

        return float(val(x))
    return float(val)

# Integrated asymmetry driver
K = np.zeros_like(rho)

# Forward Euler / trapezoid hybrid step
for i in range(1, rho.size):
    dr = rho[i] - rho[i - 1]

    gB = 0.5 * (Gamma_B[i] + Gamma_B[i - 1])
    gBb = 0.5 * (Gamma_Bbar[i] + Gamma_Bbar[i - 1])

    sB = 0.5 * (_src(source_B, rho[i]) + _src(source_B, rho[i - 1]))
    sBb = 0.5 * (_src(source_Bbar, rho[i]) + _src(source_Bbar, rho[i - 1]))

    n_B[i] = n_B[i - 1] + dr * (-gB * n_B[i - 1] + sB)
    n_Bbar[i] = n_Bbar[i - 1] + dr * (-gBb * n_Bbar[i - 1] + sBb)

    n_B[i] = max(n_B[i], 0.0)
    n_Bbar[i] = max(n_Bbar[i], 0.0)

    K[i] = K[i - 1] + dr * (delta_gamma[i] + delta_gamma[i - 1])

denom = n_B + n_Bbar
A = np.where(denom > 0, (n_B - n_Bbar) / denom, 0.0)

# Cosmology-inspired normalized asymmetry measures
n_gamma = 1.0
s = 7.04 * n_gamma
eta_photon = (n_B - n_Bbar) / n_gamma
eta_entropy = (n_B - n_Bbar) / s

out = {
    "rho": rho,
    "a": a,
    "phi": phi,
    "geom_kernel": geom_kernel,
    "delta_gamma": delta_gamma,
    "Gamma_B": Gamma_B,
    "Gamma_Bbar": Gamma_Bbar,
    "n_B": n_B,
    "n_Bbar": n_Bbar,
    "A": A,
    "K": K,
    "eta_photon": eta_photon,
    "eta_entropy": eta_entropy,
    "params": {
        "a0": a0,
        "k": k,
        "epsilon0": epsilon0,
        "alpha": alpha,
        "gamma0": gamma0,
        "delta0": delta0,
        "n0": n0,
        "a_min": a_min,
    }
}

```

```

        "phi0": phi0,
        "phi_beta": phi_beta,
        "use_phi": use_phi,
    },
}
return out if return_all else A

def closed_form_asymmetry(
    rho,
    a0=1.0,
    k=0.5,
    epsilon0=0.02,
    alpha=1.0,
    delta0=0.0,
    a_min=1e-9,
):
    """
    Closed-form A(rho) for the source-free symmetric model
    with linear cone geometry a(rho)=a0-k*rho.
    """
    rho = np.asarray(rho, dtype=float)
    a = np.maximum(a0 - k * rho, a_min)

    if np.isclose(alpha, 1.0):
        I = np.log(a0 / a) / k
    else:
        I = (a0 ** (1 - alpha) - a ** (1 - alpha)) / (k * (1 - alpha))

    K = 2.0 * epsilon0 * I
    if delta0 != 0.0:
        K = K + np.log((1 + delta0) / (1 - delta0))

    return np.tanh(0.5 * K)

```

Как использовать

Пример с чистой геометрической моделью без источников: [\[1\]](#)

```

rho = np.linspace(0.0, 1.8, 500)

res = baryon_asymmetry_model(
    rho,
    a0=1.0,
    k=0.45,
    epsilon0=0.03,
    alpha=1.0,
    gamma0=0.7,
    delta0=0.0,
)

A_final = res["A"][-1]
eta_s_final = res["eta_entropy"][-1]
print(A_final, eta_s_final)

```

Если хочешь включить вклад пятого измерения $\phi(\rho)$, ставь `use_phi=True`; тогда ядро асимметрии будет зависеть от $(a\phi)^{-\alpha}$, а не только от $a^{-\alpha}$ [1].

```
res = baryon_asymmetry_model(  
    rho,  
    a0=1.0,  
    k=0.45,  
    epsilon0=0.03,  
    alpha=1.2,  
    gamma0=0.7,  
    phi0=1.0,  
    phi_beta=0.5,  
    use_phi=True,  
)
```

Что именно моделируется

Функция реализует систему

$$\frac{dn_B}{d\rho} = -\Gamma_B(\rho)n_B + S_B(\rho), \quad \frac{dn_{\bar{B}}}{d\rho} = -\Gamma_{\bar{B}}(\rho)n_{\bar{B}} + S_{\bar{B}}(\rho),$$

где

$$\Gamma_B = \gamma_0 - \varepsilon_0 \mathcal{G}(\rho), \quad \Gamma_{\bar{B}} = \gamma_0 + \varepsilon_0 \mathcal{G}(\rho),$$

а геометрическое ядро берётся как $\mathcal{G}(\rho) = a(\rho)^{-\alpha}$ или $\mathcal{G}(\rho) = (a(\rho)\phi(\rho))^{-\alpha}$ [1]. Нормировка через $\eta = (n_B - n_{\bar{B}})/s$ тоже физически мотивирована, потому что в космологии часто используют именно отношение к энтропийной плотности, а не только “сырую” разность плотностей [1].

Что можно улучшить

Для следующего шага я бы добавил:

- решение через `scipy.integrate.solve_ivp` вместо явного шага;
- температурную переменную T вместо ρ ;
- условие “freeze-out” и связь с эффективным числом степеней свободы g_* , поскольку барионную асимметрию обычно нормируют через энтропийную плотность s [1] [2].

Хочешь, я сразу сделаю вторую версию функции, уже ближе к физически аккуратной модели с `solve_ivp`, $g_*(T)$, freeze-out и наблюдаемым η_B ?

✱✱

1. https://en.wikipedia.org/wiki/Baryon_asymmetry.

2. <https://lss.fnal.gov/archive/2020/pub/fermilab-pub-20-275-t.pdf>

3. <https://repositorio.unesp.br/server/api/core/bitstreams/c8875b5e-cee1-4001-b5ca-58b15730e80a/content>

4. <https://arxiv.org/pdf/1806.00597.pdf>

5. <https://link.aps.org/accepted/10.1103/PhysRevLett.108.221301>
6. <http://arxiv.org/pdf/2301.05722.pdf>
7. <https://arxiv.org/html/2509.13098>
8. https://www.physik.uni-wuerzburg.de/fileadmin/11030200/forschung/publikationen/Dissertationen/Banik_Amitayus_Thesis.pdf
9. <https://courseshub.world/baryogenesis/module0-cosmological-context>
10. <https://www.arxiv.org/pdf/hep-ph/9305274.pdf>